Claude

THINK
CREATE
SOLVE
TOGETHER

Claude 102

Settings, Profile, Projects, Skills, and Artifact Usage

PERSONALIZE / ORGANIZE / UNLOCK / CREATE / DO MORE

Settings
TAILOR
YOUR EXPERIENCE

Profile
MAKE IT
YOURS

Projects
ORGANIZE
YOUR WORK

Skills
EXTEND
YOUR CAPABILITIES

Artifacts
TURN IDEAS
INTO REAL THINGS

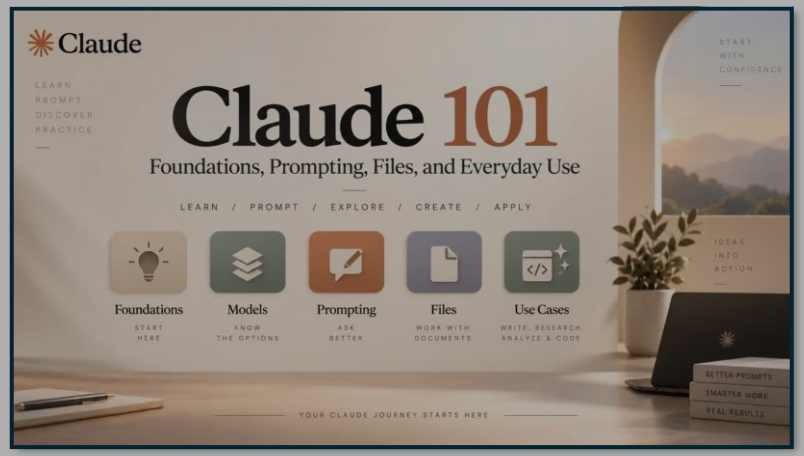
A MORE
CAPABLE
YOU

IDEAS
INTO
PROGRESS

BETTER PROMPTS
BRIGHTER IDEAS
BIGGER IMPACT

HUMAN POTENTIAL, AMPLIFIED

Three sessions, one journey







101 • Foundations

What Claude is and where it fits
Model families
Prompting and context
Documents and files

102 • Workspace

Settings, profile, and preferences
Projects: files, instructions, context
Skills: reusable workflows
Artifacts: docs, code, diagrams

103 • Advanced

Agents + sub-agents
Coordinating and delegating work
Model Context Protocol (MCP)
Automation + integrations

Agenda

- Claude 101 recap
- Settings
- Projects
- Skills
- Artifacts
- Putting it together
- Demos throughout the presentation
- Q&A



Claude 101 in 60 seconds

1 · Pick a model

Start fast and cheap; step up when quality needs it

2 · Prompt

Role, task, context, constraints, output

3 · Add files

Attach the evidence instead of paraphrasing it

4 · Verify

You own the result: check facts, numbers, and code



Context window

Everything Claude can see in one chat. Tonight we stop re-typing it every time.



Iterate

Ask, critique, refine, verify, reuse. Tonight is all about the reuse step.



The gap

Every new chat starts from zero. Settings, Projects, and Skills close that gap.

Demo



Print StartTime

Extract all dates and format them as YYYY-MM-DD in a bulleted list: 'The project kicked off on March 5th, 2024. We hit our first milestone on July 12 of the same year, and final delivery happened on Jan 15, 2025.’

When done after outputting response Print EndTime and Print ElapsedTime

Exploring Model Differences

Use new session for each run across Haiku, Sonnet, Opus, and Fable.

Settings

Customize AI behavior, privacy, memory, integrations, and terminal or web interface options

From a chat to a workspace

WORKSPACE

Each layer saves you from repeating a different kind of context.

Settings & profile

Who you are and how you like answers, in every chat

Projects

Files, instructions, and memory for one workstream

Skills

Repeatable procedures Claude loads only when needed

Artifacts

The deliverables: docs, code, diagrams, and apps

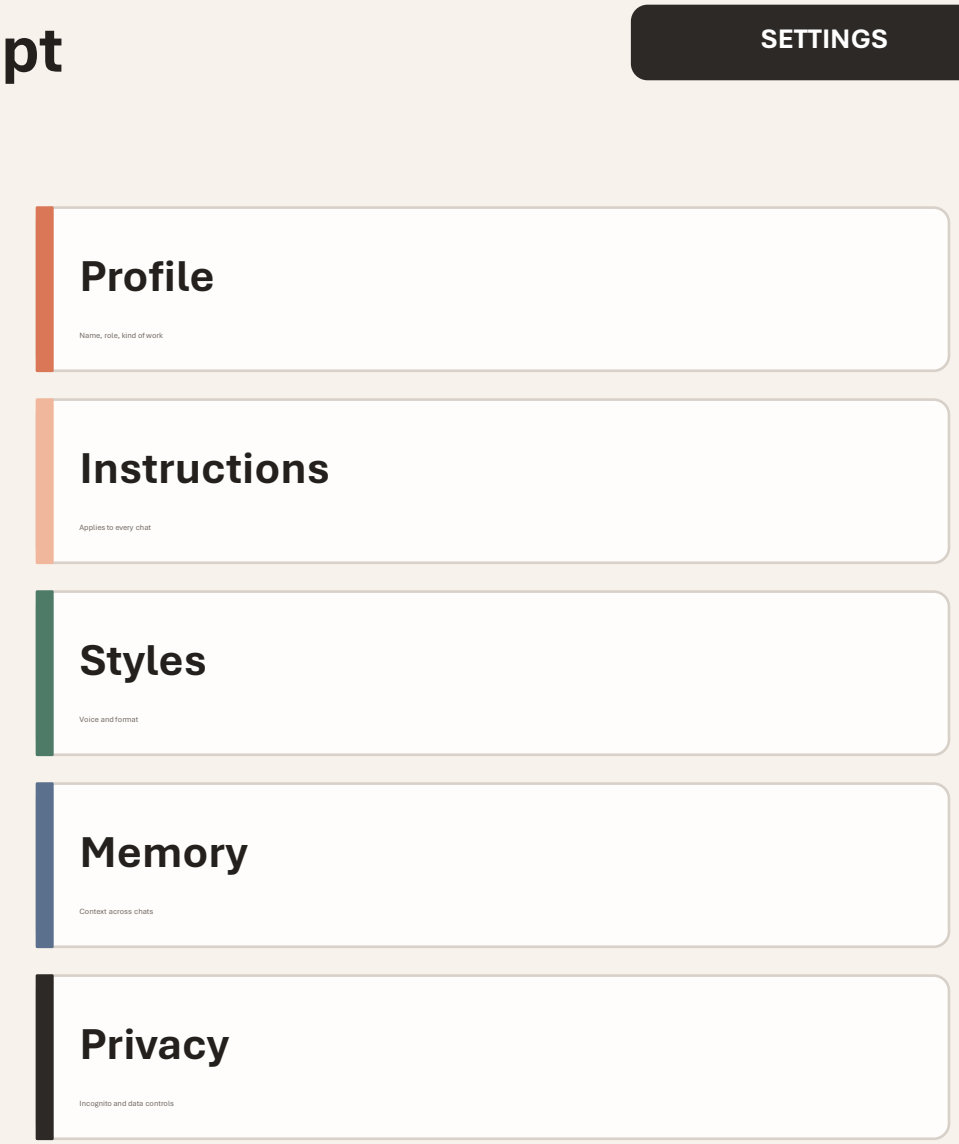
Start here ↑

101 was the chat • 102 is the workspace • 103 is the system

Settings: tell Claude once, not every prompt

Click your initials (lower left) → Settings. Skills and styles live under Customize.

- Profile: your name and what best describes your work.
- Instructions for Claude apply to every conversation you start.
- Capabilities turn on code execution, web search, and memory.
- Check capabilities first: Skills and Artifacts need code execution.



Write your instructions like a briefing

SETTINGS

Generic instructions get generic answers. Give Claude the facts it would otherwise guess.

Generic

“Be helpful and concise.”

Problems

- No role or stack
- No preferred tools
- No output defaults
- No boundaries

Specific

“I'm a .NET architect. Default to C# 12, .NET 8, and Azure. Show code first, then a short why. Prefer Polly for resilience and xUnit for tests. Flag anything that changes cost or security. Ask before assuming business rules.”

Styles change the voice, not the facts

STYLES

Pick a style for any chat, or create your own from a writing sample.

Normal

Claude's default responses

Concise

Shorter and more direct

Formal

Clear and polished

Explanatory

Teaches concepts as it answers

Custom style: upload a writing sample, or describe the voice you want, and Claude builds the style.

Memory: what Claude carries between chats

What memory does

- Summarizes past chats so new ones start with context
- Available on every plan, including Free
- Keeps a separate memory for each project
- View and edit it in Settings → Capabilities
- Say “remember that…” to update it on the spot

Where to be careful

- Review it now and then: stale facts carry forward
- Use incognito chats for one-offs you don't want kept
- Searching past chats is a paid-plan feature
- Work and personal topics mix unless you use projects
- Sensitive data: follow your company's AI policy

What the free plan gets you in the workspace

PLANS

About half of you are on Free. Everything tonight works there unless it says Paid.

5	All plans	All plans	All plans	Paid
projects on Free	memory + incognito	skills (code execution on)	create + publish artifacts	chat search, project RAG

Paid plans add project RAG (up to 10x knowledge), artifact storage, and connected apps. Team and Enterprise add sharing.

Source: Claude Help Center articles on Projects, Memory, Skills, and Artifacts, checked October 12, 2026.

DEMO A • Same question, two setups

LIVE DEMO

Scenario: personalize Claude for a .NET developer, then ask the same question again.

Step 1 • Baseline

In a fresh chat, ask:

“How should I add retries to an HttpClient call?”

Step 2 • Personalize

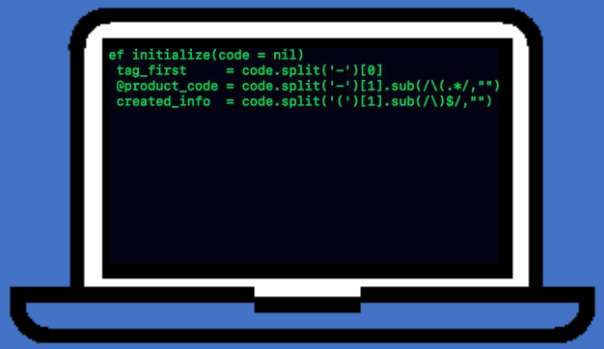
Paste profile-instructions.txt into Instructions for Claude: .NET 8, C#, Polly, code first.

Step 3 • Compare

Open a new chat, ask the same question, then switch the style to Concise.

Goal: show that one minute of settings improves every chat that follows.

Demo



Settings

Explore settings in claude.ai


Project HAIL MARY

Projects

Workspace for one stream of work, with its own knowledge, instructions, chats, and memory.

What a Project is

A Project is a workspace for one stream of work, with its own knowledge, instructions, chats, and memory.

Every chat in the project starts with your files and instructions.

Available on every plan. Free accounts can create up to five.

Team and Enterprise plans can share projects with colleagues.



Instructions

How to behave here



Knowledge

Files Claude can search



Chats

History in one place



Memory

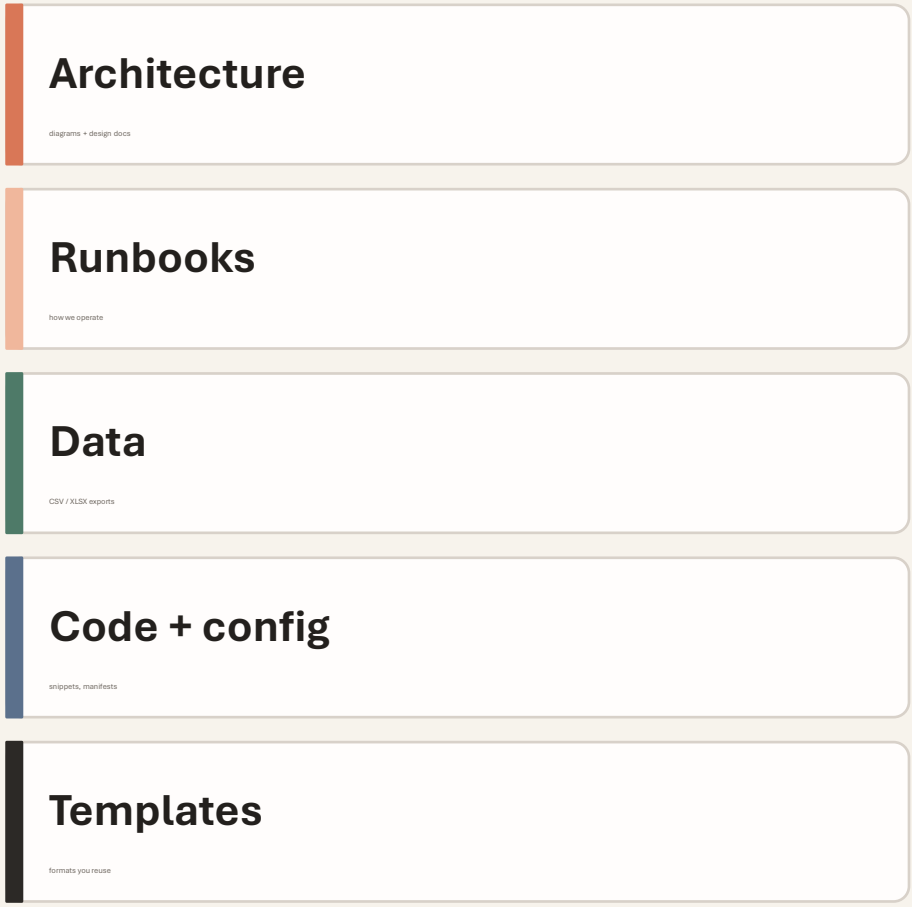
Separate per project

Project knowledge: upload the evidence once

PROJECTS

Every chat in the project can draw on it without re-attaching anything.

- Upload docs, code, data, and notes Claude should always have.
- Ask Claude to cite the file and section behind each answer.
- Paid plans switch to retrieval (RAG) as knowledge grows, up to 10x capacity.
- Replace files when they change: stale knowledge, stale answers.



Project instructions reuse the five-part prompt

PROJECTS

Write the prompt you'd otherwise repeat in every chat of this project.

1	**ROLE / POV**	“You are the SRE assistant for the Contoso Orders API…”
2	**TASK**	“Help triage incidents, explain costs, review changes…”
3	**CONTEXT**	“.NET 8 on AKS, Azure SQL, 99.9% SLO, p95 < 500 ms…”
4	**CONSTRAINTS**	“Use only project files; say when data is missing…”
5	**OUTPUT**	“Cite the file behind each claim; end with next checks.”

Keep instructions short and stable. Facts that change belong in knowledge files.

Chat, incognito, or Project?

PROJECTS

Pick by how often you'll come back, and use the lightest container that keeps the context.

Plain chat

One-off question or quick draft. Memory may still add helpful context.

Incognito chat

Sensitive or throwaway. Not saved to memory and not included in chat search.

Project

Recurring work with shared files and instructions: a service, a client, a course.

Shared project

Team knowledge on Team and Enterprise plans, with view or edit access per person.

Project hygiene keeps answers sharp

PROJECTS

A cluttered project behaves like a long, unfocused chat.

One workstream

One project per service, client, or goal

Fresh files

Replace outdated docs; don't stack versions

Short chats

New chat per task; the project keeps context

Review memory

Check the project's memory summary now and then

Rule of thumb: if you paste the same context twice, it belongs in the project.

DEMO B • Build the Contoso Ops project

LIVE DEMO

Scenario: one project that knows the Contoso Orders API, from architecture to costs.

Files

architecture.md • runbook.md
cloud-costs.csv • change-log.txt

Upload once. Every chat in the project can use them.

Ask

“What changed before the 14:05 latency spike, and what does the runbook say to check first?”

Verify

Start a second chat with no pasting. Require file and row citations for every claim.

Demo



Projects

Polling the audience


I don’t have any good skills

Skills

Modular instruction packages, scripts, and resources that teach Claude how to perform specialized, repeatable tasks

How a Skill works

1 · Describe

Each skill has a name and a one-line description

2 · Match

Your request matches a skill's description

3 · Load

Only then does Claude read the SKILL.md

4 · Run

Instructions, scripts, and templates do the work



Progressive disclosure

Many skills cost almost nothing until one is needed, so context stays focused.



Built-in skills

Word, Excel, PowerPoint, and PDF creation come ready to use, automatically.



Code execution on

Skills run in Claude's sandbox. Turn on code execution in Settings.

Anatomy of a custom skill

SKILLS

A folder with a SKILL.md file, zipped and uploaded under Customize → Skills.

```
---
name: incident-postmortem
description: Turns incident notes into a
  blameless postmortem. Use for incident reviews.
---
# Incident postmortem
1. Build a UTC timeline from the notes.
2. Separate facts from hypotheses.
3. Follow postmortem-template.md.
4. Give each action item an owner.

Folder:
  SKILL.md
  postmortem-template.md
```

- The description decides when Claude uses the skill.
- Say when to use it: “Use for…”
- Write steps Claude can actually follow.
- Bundle the templates the steps refer to.

Skill Authoring Best Practices

1. Avoid generic vague or reserved words for the skill name
   i.e. helper, utils, tools
2. Describe what and when. List key points first in the skill description.
3. Avoid explain the obvious
   i.e. a paragraph explaining what a PDF is. Claude already knows this.
4. Progressive Disclosure
   SKILLS.md < 500 lines
   Keep references 1 level deep
   If reference file > 100 lines use table of contents
5. Test skill against models you plan to use. Skills add as additions to model
   Claude Haiku - fast, economical
   Claude Sonnet - balanced
   Claude Opus - powerful reasoning - Does the skill avoid over-explaining

Demo


ef initialize(code = nil)
tag_first = code.split('-')[0]
@product_code = code.split('-')[1].sub(/\(.*/,"")
created_info = code.split('(')[1].sub(/\)$/,"")

Skills





Artifacts

Standalone piece of work in its own panel that you can iterate on, download, or share.

What an Artifact is

An artifact is a standalone piece of work in its own panel that you can iterate on, download, or share.

Created on every plan once code execution is on.

Each change makes a new version you can step back to.

Published artifacts work for viewers without an account.



Documents

Reports, guides, notes



Diagrams

Flowcharts, architecture



Dashboards

Charts from your data



Code + apps

Tools, sites, prototypes

Sharing and publishing artifacts

How to share

- Free, Pro, and Max: Publish for a public link
- Team and Enterprise: share inside your organization
- Viewers don't need a Claude account to use it
- Copy an artifact to remix it in your own chat
- AI-powered artifacts run on each viewer's own plan

Before you share

- Check which version is showing before you publish
- Shared storage is visible to everyone using it
- Unpublishing is permanent and deletes stored data
- Only remix artifacts from people you trust
- Keep secrets and customer data out of public links

DEMO D • Project + Artifact

LIVE DEMO

Scenario: from incident data to a shareable dashboard, inside the Contoso Ops project.

Project

No context to paste. Start a new chat in the project.

Artifact

Using the attached Titanic dataset ZIP file, create an interactive **Titanic Survival Insights Dashboard** as a Claude Artifact.

Include key statistics, survival rates, and visualizations comparing passengers by age, gender, passenger class, and embarkation port.

Add interactive filters that automatically update the charts and statistics.

Use a professional, Power BI-style dashboard with KPI cards, bar charts, and donut charts. Make it visually appealing and easy to explore.
Use actual data from the ZIP file for all calculations.

Share

Iterate once, publish, open the link on a phone, and check one number against the CSV.

Demo



Artifacts

Titanic Survival Insights Dashboard

Summary

When to use what

Which tool for which job?

TOGETHER

Each one removes a different kind of repetition.

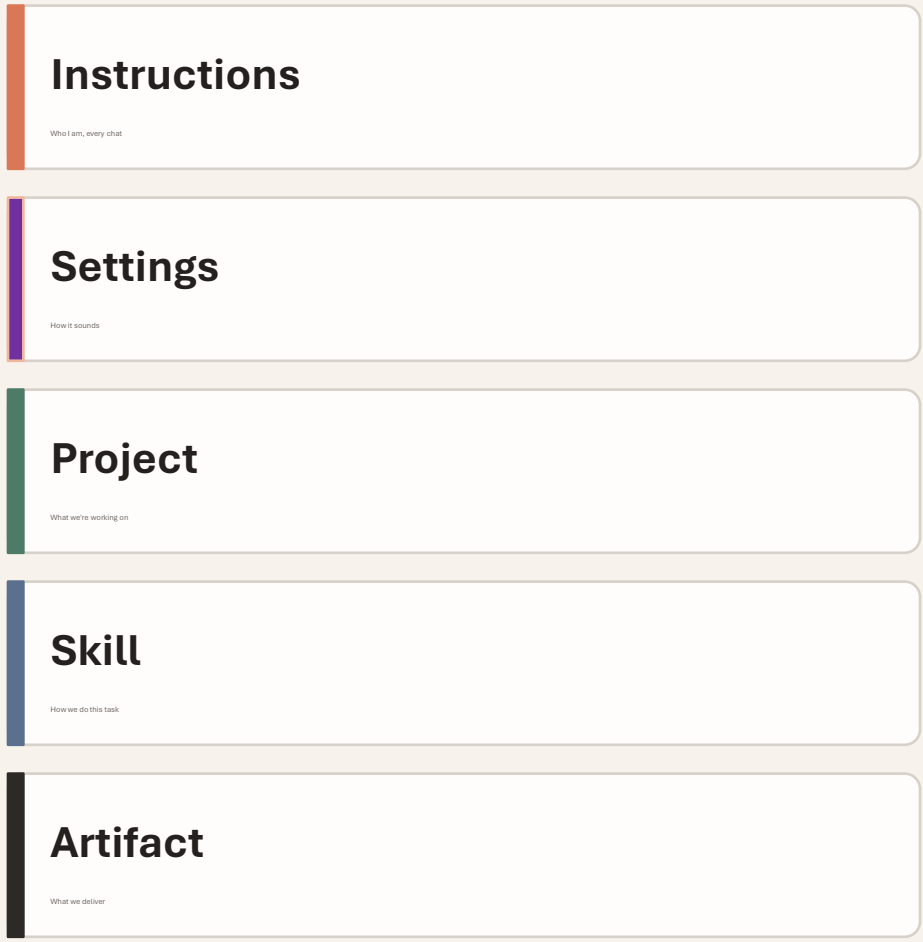
Instructions shape every chat.

Settings set account-wide instructions, communication preferences, and domain context across conversations.

Projects hold the context for one stream of work.

Skills hold procedures you repeat across projects.

Artifacts are what you hand to someone else.



Six practical tips

1 Start with your profile

```
Instructions: .NET 8, C#, code first, flag
cost and security.
```

2 One project per workstream

```
Contoso Ops: architecture, runbook, costs,
change log.
```

3 Ask for the artifact by name

```
Build an interactive dashboard from this CSV.
```

4 Let Claude write the skill

```
Turn this template into a skill. Interview me
first.
```

5 Start fresh chats often

```
New task, new chat. The project keeps the
context.
```

6 Check capabilities first

```
Settings → Capabilities: code execution,
memory.
```

Claude 102 – Key Takeaways

- Set your profile and instructions once
- Use a project for each recurring workstream
  - Keep its files current
- Turn repeated steps into skills
- Treat artifacts as deliverables: iterate, verify, share





References

Daniel Colón LinkedIn
https://www.linkedin.com/in/danielecolon/

Presentation Slides/Code
https://github.com/danielecolon/Claude

Get started with Claude
https://support.claude.com/en/collections/9811058-get-started-with-claude

Projects
https://support.claude.com/en/articles/9517075-what-are-projects

Use skills in Claude
https://support.claude.com/en/articles/12512180-use-skills-in-claude

Artifacts
https://support.claude.com/en/articles/9487310-what-are-artifacts-and-how-do-i-use-them

Memory
https://support.claude.com/en/articles/11817273